



Clean Architecture & Cubit Guidelines

مبني على خبرة شغل حقيقي، هدفه تنظيم التفكير قبل كتابة أي **ملخص عملي** هذا المستند عبارة عن وتسجيل التطوير والصيانة، bugs جديدة، وتقليل الـ Feature.

فلسفة الشغل

- وخذ بريك مهما كان وصلت لإيه **ساعتين ساعتين** قسم شغلك.
- وطريقتك اللي مرتاح فيها **خليك بنفس كودك**، مش كل حاجة ليها قواعد ثابتة.
- البساطة + التنظيم = كود قوي وسهل التعديل.



Architecture Overview

```
UI
├── Cubit
│   └── UseCase
│       ├── Repository (Abstraction)
│       │   └── Repository (Implementation)
│       │       └── DataSource (Remote / Local)
```



DOMAIN LAYER



Entity

- يتمثل الداتا اللي الدومين محتاجها فقط.
- JSON أو Firebase أو API مفيش أي منطق خاص بالـ.



Repository (Abstraction)

- بين الدومين والداتا Contract.
- مفيش أي تفاصيل تنفيذ.



UseCase

- Repository Abstraction من الـ methods بيستدعي.
- حالة استخدام واحدة واضحة = UseCase كل.

DATA LAYER

Model

- من غير لعب الموديل زي ما الريموت باعت.
- هنا default values مفيش.

Mapper

- Model → Entity مسؤول عن التحويل من.
- هنا بس نستخدم:
- null safety
- default values

Repository Implementation

- Repository Abstraction بيطبق الـ.
- DataSource بيعتمد على.

DataSource

- Remote (API / Firebase)
- Local (Cache / DB)

Service Locator (Locator)

⚠️ الترتيب مهم جدًا!

ربط Abstraction بـ Implementation

```
Sl.registerLazySingleton<Repo>(() => RepoImpl(data: Sl()));
```

تكون متسجلة قبلها data لازم الـ.

Dependency Inversion

- Implementation مش Abstraction دايمًا اعتمد على.

```
class DataSource {
  final Repo repo;
  DataSource({required this.repo});
}
```

LocatorApp مفيش داعي لملف

- لو المشروع مش كبير locator global خلي الـ

Cubit Guidelines

BlocProvider

✓ استخدم `create` مش `value`

```
BlocProvider(
  create: (context) => Sl<MyCubit>(),
  child: Container(),
)
```

Access Cubit

```
final cubit = context.read<MyCubit>();
```

States

- **State لازم تدخل** (API / Firebase) أي داتا جاية من برا
- فقط State يعتمد على UI الـ

مهمة جدًا Notes

Error Handling

- في `try / catch` دايماً استخدم
- Cubit
- Services

```
try {
    // logic
} catch (e) {
    emit(ErrorState());
    return; // يكسر الدالة
}
```

State Usage

```
final currentState = authCubit.state;
```

أكثر من مرة state مفيد لما تحتاج نفس الـ -

Callback Functions

```
void sayHello() {
    print("Hello");
}

void sayName(String name) {
    print("Hello $name");
}
```

```
onPressed: sayHello,
onPressed: () => sayName("Mohamed"),
```

Mapper Rule (قاعدة ذهبية)

-  default values مفيش في Model
-  Entity مفيش تعديل في
-  الماير هو المسؤول عن:
- null handling
- default values

Reference

- Tharwat Samy – Fruits Hub
-

Checklist **جديدة Feature قبل أي**

- [] Domain واضح
 - [] Repository Abstraction موجود
 - [] UseCase حالة لكل واحد
 - [] Mapper عن التحويل
 - [] try / catch في Cubit
 - [] فقط State يعتمد على UI
-

الكود النظيف مش في التعقيد، في التنظيم والاستمرارية: **خلاصة #**